

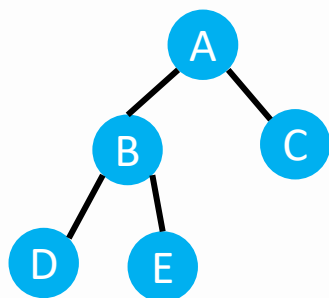
树的遍历

树的遍历方法有深度优先遍历和广度优先遍历，遍历方法和搜索类似。

```
void dfs(int u,int fa)
{
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
    }
}
```

树的DFS序

当我们对树进行dfs遍历的时候，对于每个结点，我们在**进入**和**出去**的时候都记录一下该点的编号，这样我们得到的长度为 $2N$ 的序列就是树的DFS序。



树的DFS遍历：ABDEC

树的DFS序：ABDDEEBCCA

树的DFS序的作用：我们会发现在一棵树中，以结点 i 为根结点的子树在DFS序中就是以 i 为起点， i 为终点的一段区间。比如BDDEEB就表示以B为根结点的子树，每个结点刚好都被算了两次。

树的深度

树上的点到该树的根经过的简单路径最多的节点数

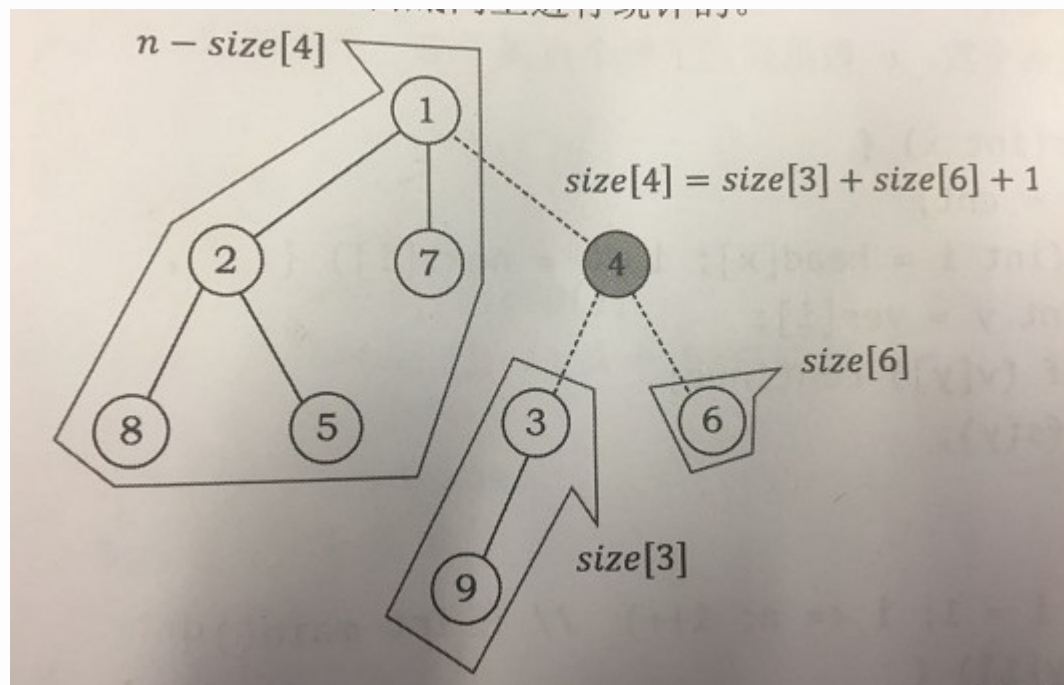
```
d[root]=1;
void dfs(int u,int fa)
{
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            deep[v]=deep[u]+1;
            dfs(v,u);
        }
    }
}
```

树的节点数

```
void dfs(int u,int fa)
{
    size[u]=1;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            dfs(v,u);
            size[u]+=size[v];
        }
    }
}
```

树的重心

找到一个点提根，其所有子树中最大的子树结点数量最少，这个点就是这颗树的重心，删除重心后，生成的多棵树尽可能平衡。



树的重心

```
void dfs(int u,int fa)
{
    size[u]=1;
    int Max=0;
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
        {
            dfs(v,u);
            size[u]+=size[v];
            Max=max(Max,size[v]);
        }
    }
    Max=max(Max,n-size[u]);
    if(Max<ans)
    {
        ans=Max;
        pos=u;
    }
    else if(Max==ans&&pos>u)
        pos=u; //选择编号最小的
}
```

树的重心

- 1: 树的重心的每棵子树的大小一定小于等于 $n/2$
- 2: 每棵子树大小都小于等于 $n/2$ 的点一定是这棵树的重心
- 3: 如果树的重心不止一个, 那么最多两个, 并且相邻
- 4: 树中所有点到某个点的距离和中, 到重心的距离和最小, 如果有两个重心, 他们的距离一样。
- 5: 两棵树通过一条边相连成为一棵新树, 新树的重心一定在原来两棵树的重心的路径上
- 6: 如果一棵树删除或者增加一个结点, 重心最多移动一个结点

会议

题目描述

 展开

有一个村庄居住着 n 个村民，有 $n-1$ 条路径使得这 n 个村民的家联通，每条路径的长度都为1。现在村长希望在某个村民家中召开一场会议，村长希望所有村民到会议地点的距离之和最小，那么村长应该要把会议地点设置在哪个村民的家中，并且这个距离总和最小是多少？若有多个节点都满足条件，则选择节点编号最小的那个点。

会议

树的重心裸题，如果直接树的重心的性质，就可以直接求树的重心就可以了。

如果不知道，一个简单的树上递推也可以算出来，考虑其他节点到父亲节点和儿子的距离的变化量，分为三种情况，父亲节点上面，儿子所在子树，父亲节点的其他子树

医院设置

题目描述

展开

设有一棵二叉树，如图：



其中，圈中的数字表示结点中居民的人口。圈边上数字表示结点编号，现在要求在某个结点上建立一个医院，使所有居民所走的路程之和为最小，同时约定，相邻接点之间的距离为 1。如上图，若医院建在 1 处，则距离和 $= 4 + 12 + 2 \times 20 + 2 \times 40 = 136$ ；若医院建在 3 处，则距离和 $= 4 \times 2 + 13 + 20 + 40 = 81$ 。

医院设置

该题是一道带权的树的重心的题，直接在树上递推就可以了：设 v 是 u 的一个儿子，那么：

对于 u 以及 u 上方的结点以及 u 的其他子树，所有结点到重心结点距离都增加1，对于 v 下面的结点，所有结点到重心的距离都减少1，那么答案就是：

$$dp[v] = dp[u] + (size[根] - size[v]) - size[v];$$

树的双中心

题目描述

展开

给定一棵树 $T=(V,E)$ ，其中 V 为节点集合， E 为边集合。对于 V 中的每个节点 v ，有一个权值函数 $W(v)$ ，该函数的值均为正整数。记 $d(u,v)$ 为节点 u 和 v 之间的距离，表示它们之间唯一的一条路径的边数。若 u 和 v 为同一个节点，则 $d(u,v)=0$ 。你的任务是找出两个不同的节点 x 和 y ，使得以下表达式 $S(x,y)$ 的值最小


$$S(x,y) = \sum_{v \in V} (W(v) \cdot \min \{d(v,x), d(v,y)\})$$

输入格式

第一行为 N ， $1 < N \leq 50000$ ，表示树的节点数目，树的节点从 1 到 N 编号。接下来 $N-1$ 行，每行两个整数 U, V ，表示 U 与 V 之间有一条边。再接下 N 行，每行一个正整数，其中第 i 行的正整数表示编号为 i 的节点权值为 $W(i)$ ，树的深度 ≤ 100

树的双中心

首先，要看明白题目的意思，题目的意思是从树上选择两个点 x , y ，则可以把树分成两个集合 A , B ，使得 A 集合所有点都去 x ， B 集合所有点都去 y ，而这两个集合的分界点就是树上的一条边。

所以一种很简单的暴力方法就是枚举分界边是哪一条边，然后 $O(n)$ 的去求树的重心，但是时间复杂度为 $O(n^2)$ ，过不了。

树的双中心

我们考虑下枚举边的时候的转移，首先，带权树的重心求法我们已经知道了：

$$dp[v] = dp[u] + size[1] - 2 * size[v];$$

根据公式，只有当 $size[v] * 2 > size[1]$ 的时候，答案才会更优，很明显，对于任意一个结点的儿子结点，满足这个条件的结点数 ≤ 1 。

所以我们找树的重心的时候只需要验证结点数最大的儿子转移就可以了。

树的双中心

所以我们需要记录每个结点数量和每个结点最大的儿子，但是由于有删边操作，如果删的那条边刚好是结点数最大的儿子那条边怎么办？那么此时就是节点数次大的儿子也是可能是新的重心的，所以我们维护一个最大值和次大值。

因为题目中说了树的深度不会超过100，所以该题的时间复杂度应该是树的节点数*树的深度，所以是可以过的。

树的直径

树中任意两个结点的简单路径的距离的最大值。

求解方法：

先从任意一个点跑一遍dfs，走到离当前起点最远的点（如有多个，随便选择一个），该点就是树的直径的一个端点，再从这个最远的点跑一遍dfs，离当前起点最远的点就是直径的另一个端点，他们之间的距离就是树的直径。

证明：

如果起点 u 在树的直径上，最远的点 v 一定也在直径上（否则他就不是最远的点）；如果起点 u 不在树的直径上，那么 $u-v$ 一定和树的直径相交，交点一定为 v ，否则 $u-v$ 不可能最长，其中一部分替换为直径的另一部分更长。

树的直径

树的直径的性质：

1: 树上任意一点到树的最远距离就是该点到树的直径的其中一个端点的距离。

2: 如果一棵树有多条直径，这些直径交于一点，并且该点是直径的中点。

3: 两棵树用一条边相连，新树的直径的两端一定是原本两棵树直径四个端点中的两个。

4: 边权非负可以用贪心和dp求解，但是如果边权可正可负只可以用dp求解，维护一个最长链和次长链

树的直径

给你一颗N个节点的树，求每个点到其他点的最大距离。

输入：

5

1 2

1 3

1 4

4 5

输出：

2 3 3 2 3

$N \leq 500000$

MSOJ 1345 最长链

逃学的小孩

题目描述

展开

Chris家的电话铃响起了，里面传出了Chris的老师焦急的声音：“喂，是Chris的家长吗？你们的孩子又没来上课，不想参加考试了吗？”一听说要考试，Chris的父母就心急如焚，他们决定在尽量短的时间内找到Chris。他们告诉Chris的老师：“根据以往的经验，Chris现在必然躲在朋友Shermie或Yashiro家里偷玩《拳皇》游戏。现在，我们就从家出发去找Chris，一旦找到，我们立刻给您打电话。”说完砰的一声把电话挂了。

Chris居住的城市由N个居住点和若干条连接居住点的双向街道组成，经过街道x需花费Tx分钟。可以保证，任两个居住点间有且仅有一条通路。Chris家在点C，Shermie和Yashiro分别住在点A和点B。Chris的老师 and Chris的父母都有城市地图，但Chris的父母知道点A、B、C的具体位置而Chris的老师不知。

为了尽快找到Chris，Chris的父母会遵守以下两条规则：

1. 如果A距离C比B距离C近，那么Chris的父母先去Shermie家寻找Chris，如果找不到，Chris的父母再去Yashiro家；反之亦然。
2. Chris的父母总沿着两点间唯一的通路行走。

显然，Chris的老师知道Chris的父母在寻找Chris的过程中会遵守以上两条规则，但由于他并不知道A，B，C的具体位置，所以现在他希望告诉你，最坏情况下Chris的父母要耗费多长时间才能找到Chris？

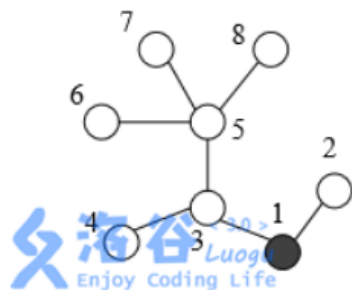
逃学的小孩

我们通过画图和分析可以很容易判断出，这个小孩可能在的位置就是树的直径的两个端点，这样才能保证树上任意一个点到这两点的距离之和最大，证明可以用反证法。

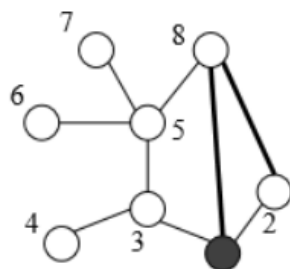
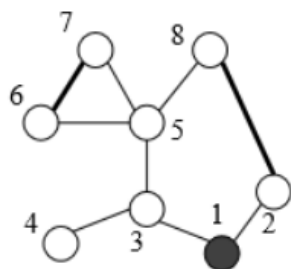
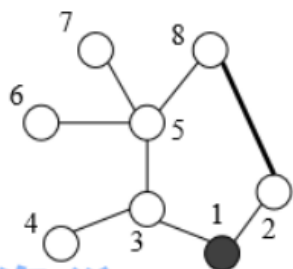
接下来就要去找一个点C，我们的移动路线一定是先从家到其中一个点(B或C)，再到另一个点，也就是说无论什么情况一定有一条路径是直径，对于另一条路径，就是树上的点到至今两个端点的距离的较小值的最大值

巡逻

在一个地区中有 n 个村庄，编号为 $1, 2, \dots, n$ 。有 $n - 1$ 条道路连接着这些村庄，每条道路刚好连接两个村庄，从任何一个村庄，都可以通过这些道路到达其他任一村庄。每条道路的长度均为 1 个单位。为保证该地区的安全，巡警车每天要到所有的道路上巡逻。警察局设在编号为 1 的村庄里，每天巡警车总是从警察局出发，最终又回到警察局。下图表示一个有 8 个村庄的地区，其中村庄用圆表示（其中村庄 1 用黑色的圆表示），道路是连接这些圆的线段。为了遍历所有的道路，巡警车需要走的距离为 14 个单位，每条道路都需要经过两次。



为了减少总的巡逻距离，该地区准备在这些村庄之间建立 K 条新的道路，每条新道路可以连接任意两个村庄。两条新道路可以在同一个村庄会合或结束（见下面的图例 (c)）。一条新道路甚至可以是一个环，即，其两端连接到同一个村庄。由于资金有限， K 只能是 1 或 2。同时，为了不浪费资金，每天巡警车必须经过新建的道路正好一次。下图给出了一些建立新道路的例子：



巡逻

如果 $K=1$ ，表示只修一条路，那么形成了一个环，环上的边只需要走一遍，其他边还是走两遍，那么省下的距离就是这条路连接的两点的初始路径长度-1，所以很明显，就连接直径。

但是当 $K=2$ 的时候，我们又该如何选择呢？会再形成一个环，那么这个环和第一个环有两种关系：

- (1) 相交
- (2) 不相交

巡逻

如果不相交，那么处理起来很简单，忽略加入的第一条边所成的环的路径，然后再求一次树的直径。

但是如果相交呢？根据题目要求，新建的道路必须经过正好一次。如果一条边在A不在B，那么只用走一次，一条边在A又在B，那么肯定要走两次，也就是说第一次造路并没有帮重复的边剩下路径，所以我们把第一个环的路径全变为-1（不能删除），那么再求一遍直径就可以了，注意这里只能用树形dp。

树的中心

对于一棵树，定义 $r(x)$ 为离 x 最远的点到 x 的距离，树的重心为满足 $r(x)$ 最小的点 x 。

1: 所有的直径都经过树的中心

2: 中心是任意一条直径的中点，对于直径上有偶数个点的情况存在两个中心。

3: 树上任意一点到树上的最远距离一定经过其中一个中心

城市

题目描述

展开

从加里敦大学城市规划专业毕业的小明来到了一个地区城市规划局工作。这个地区一共有 n 座城市， $n - 1$ 条高速公路，保证了任意两座城市之间都可以通过高速公路相互可达，但是通过一条高速公路需要收取一定的交通费用。小明对这个地区深入研究后，觉得这个地区的交通费用太贵。

小明想彻底改造这个地区，但是由于上司给他的资源有限，因而小明现在只能对一条高速公路进行改造，改造的方式就是去掉一条高速公路，并且重新修建一条一样的高速公路（即交通费用一样），使得这个地区的两个城市之间的最大交通费用最小（即使得交通费用最大的两座城市之间的交通费用最小），并且保证修建完之后任意两座城市相互可达。如果你是小明，你怎么解决这个问题？



城市

简单来说就删除树上的一条边，然后再连接一条边，最后让新生成的树的直径最短。

对于两棵树的合并，如何才能连边才能让合并之后的直径最短呢？

我们考虑树的中心，树的中心到树上其他点的最远距离最最近，那么如果连接两棵树的中心，那么得到的树的直径是最短的。

那么该题就转换成如何快速求树的中心。



城市

首先，我们枚举断边，对于一棵有根树而言，直径的一个端点一定是叶子结点，所以我们记录每一个点到叶子结点的最大距离。因为直径可能是最大值+次大值，所以我们再维护每一个点倒也自己结点的次大距离。因为我们每次删边之后变成了两棵树，所以我们还要求该边上面一棵树的直径，所以我们再维护一个该结点向上能到达的最远距离。

我们设 $d1$ 为向下的最长路径， $d2$ 为向下的次长路径， up 为向上的路径。设 v 是 u 的儿子结点：

$$d1[u] = \max(d1[v] + \text{edge}(u-v))$$

$d2[u]$ = 如果 $d1[u]$ 更新，则为更新前的值，如果 $d1[u]$ 不更新，那么判断这条边和 $d2[u]$ 的大小。

顺便记录一下 $d1, d2$ 从哪里转移过来

城市

关键是up数组应该如何更新呢？Up数组我们可以从跟根结点开始向下求，这和前两个数组不一样。

同样，v的父亲是u

分两种情况讨论：

(一) 如果 $d1[u]$ 不是由v转移过来的，那么
 $up[v] = \max(d1[u], up[u]) + edge(u, v)$

(二) 如果是，那么就应该和次大边比较
 $up[v] = \max(d2[u], up[u]) + edge(u, v)$

那么删除一条边之后，即 $u-v$ ，两颗树的直径分别为 $d1[v]+d2[v]$ 和 $(d1[u] \text{ 或者 } d2[u], \text{ 判断 } v \text{ 是否在 } u \text{ 的向下最长上}) + up[u]$ 。

那么最后的答案有3种情况：直径一，直径二，半径一+半径二+枚举的边



练习题

洛谷	1395	会议
洛谷	2726	树的双中心
洛谷	4408	逃学的小孩
洛谷	3629	巡逻
洛谷	3761	城市
洛谷	5658	括号树

